Associative Memory Is the Model

jfg
fourierproject@gmail.com

Abstract

Transformers and state space models (e.g. Mamba) use sophisticated mechanisms to mix information across sequence positions. We argue that **associative memory does most of the work**, and the backbone needs only minimal local mixing. We augment language models with a Hebbian memory—a matrix of outer-product associations updated at each token—and systematically ablate the backbone. On The Stack (code), at 100M parameters: Mamba alone achieves 2.53 nats; adding Hebbian memory improves it to 1.82 nats; replacing Mamba's recurrence with a 4-token depthwise convolution gives 1.79 nats with no recurrence at all. At 124M parameters on FineWeb (natural language), Conv+Heb reaches 3.39 nats versus the nanoGPT 124M baseline of 3.28 nats—a small-scale gap that linear attention variants like RetNet close and surpass at larger scales. The sequence model backbone contributes only local feature extraction; **the associative memory handles nearly all long-range recall**.

1 Introduction

Transformers [Vaswani et al., 2017] and state space models like Mamba [Gu and Dao, 2024, Dao and Gu, 2024] use complex mechanisms—quadratic attention, selective gating, discretized recurrence—to capture long-range dependencies in sequences.

But what if associative memory, not the sequence model, is the primary mechanism for long-range reasoning? In this work, we decompose language modeling into two capabilities: first, **associative binding**—storing and retrieving associations across arbitrary distances—and second, **local feature extraction**—combining adjacent tokens into useful representations.

A Hebbian associative memory [Hebb, 1949]—a decaying outer-product matrix updated at each token—handles the first almost entirely. For the second, a depthwise convolution over just 4 tokens suffices. The resulting architecture contains no recurrence, no attention, and no gating beyond SiLU, yet matches SSM-based models at comparable parameter counts and comes within 0.11 nats of nanoGPT on natural language.

Our key finding is stark: replacing Mamba with a 4-token convolution matches performance (1.79 vs 1.82 nats) at a fraction of the complexity, and a model with *no* sequence mixing at all (pure linear projection + Hebbian) still converges to 3.18 nats. On FineWeb at 124M parameters, Conv+Heb reaches 3.39 nats versus the nanoGPT baseline of 3.28—a gap consistent with other linear attention variants at this scale, which close at larger scales [Sun et al., 2023].

The associative memory is not merely an augmentation—it is nearly the entire model.

2 Method

The model (Figure 1) stacks L identical layers with pre-norm residual connections:

$$\mathbf{x}^{(l)} = \mathbf{x}^{(l-1)} + \text{HebbianMemory}(\text{Conv}(\text{RMSNorm}(\mathbf{x}^{(l-1)}))) \tag{1}$$

Preprint. Under review.

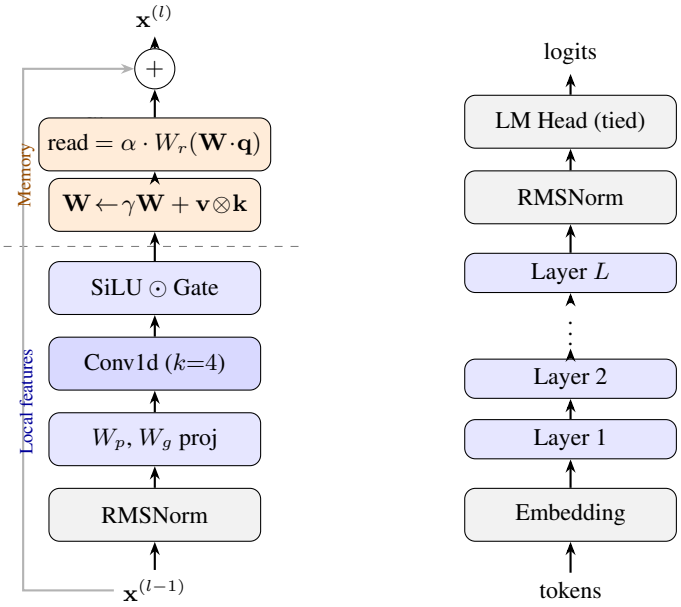

Figure 1: Architecture overview. **Left**: A single layer. The conv block (blue) extracts local features over 4 tokens. The Hebbian memory (orange) writes and reads associative bindings via a $D \times D$ outer-product matrix. **Right**: The full model stacks L identical layers with tied embeddings.

Each layer contains a local feature extractor (depthwise convolution) and a Hebbian associative memory. Token embeddings are tied with the output projection.

Parameter count For our 100M model (L=12, D=1024, E=2, d_{conv}=4), per layer:

- Conv block: $3ED^2 + ED \cdot d_{\text{conv}} \approx 6.3$M (projections + conv kernel)
- Hebbian memory: $2D^2 + 2 \approx 2.1$M (write/read projections + decay/alpha)
- Total per layer: $\sim$8.4M; 12 layers: $\sim$100M

2.1 Hebbian Associative Memory

At each position t, the model maintains a memory matrix $\mathbf{W}_t \in \mathbb{R}^{D \times D}$ that accumulates outer-product associations between consecutive representations:

$$\mathbf{W}_t = \gamma \mathbf{W}_{t-1} + \mathbf{v}_t \otimes \mathbf{k}_{t-1} \tag{2}$$

$$\text{read}_t = \mathbf{W}_{t-1} \cdot \mathbf{q}_t \tag{3}$$

where $\mathbf{v}_t = W_v \mathbf{x}_t$ is the value projection, $\mathbf{k}_t = \mathbf{x}_t$ is the write key (the raw hidden state), $\mathbf{q}_t = \mathbf{x}_t$ is the read query, and $\gamma = \sigma(\delta)$ is a learned decay with δ initialized to 4.6 ($\gamma \approx 0.99$). Here $\mathbf{x}_t$ denotes the output of the conv block at position t. The layer output combines the conv output with the memory read:

$$\mathbf{y}_t = \mathbf{x}_t + \alpha \cdot W_r \cdot \text{read}_t \tag{4}$$

where $\alpha = \exp(\log \alpha_0)$ is a learned per-layer memory strength (initialized at 0.03) and W_r is a learned read projection. A residual connection from the layer input is added on top. Both α and γ are learned per layer.

For training, we use a chunkwise parallel form with chunk size C, running in $O(TC \cdot D + T \cdot D^2)$ time; at inference, the recurrent form (Equations 2–3) runs in $O(D^2)$ per step.

2.2 Convolution Backbone

Each layer's local feature extractor is a gated depthwise convolution:

$$\mathbf{x}' = \text{SiLU}(\text{Conv1d}(W_p\mathbf{x})) \odot \text{SiLU}(W_g\mathbf{x}) \tag{5}$$

$$\text{out} = W_o\mathbf{x}' \tag{6}$$

where $W_p, W_g \in \mathbb{R}^{D\times ED}$ project to an expanded dimension (E=2), Conv1d is a causal depthwise convolution with kernel size d_{conv}=4, and $W_o \in \mathbb{R}^{ED\times D}$ projects back. This block sees exactly d_{conv} tokens of local context. All long-range information flows through the Hebbian memory.

3 Experiments

All models are $\sim$100M parameters (d_{model}=1024), trained on The Stack for 4000 steps with sequence length 2048, batch size 1 (no gradient accumulation), cosine schedule with peak 3×10^{-4} and 500-step warmup. Layer counts vary to match parameter budgets (11–15 layers; see Table 1). We report average validation loss over steps 2000–4000 in nats (natural log cross-entropy). Validation is noisy at batch size 1, so we average across eval points.

3.1 Backbone Ablation

Our central experiment replaces the backbone while keeping the Hebbian memory identical. Both use learned decay γ (init 0.99); Mamba+Heb uses a fixed α=0.04, Conv+Heb uses learned per-layer α (init 0.03). Table 1 shows results.

Table 1: Backbone ablation on The Stack ($\sim$100M params, 4000 steps).

Model	Layers	Params	Avg Val Loss
Conv(4) + Heb	12	102.9M	1.79
Mamba + Heb	11	98.5M	1.82
Mamba (no Heb)	15	102.1M	2.53

Hebbian is the dominant mechanism. Removing Hebbian memory from Mamba degrades performance from 1.82 to 2.53 nats (+0.71). The memory contributes far more than the backbone.

Conv matches Mamba with no recurrence. Conv+Heb achieves 1.79 nats, slightly better than Mamba+Heb at 1.82 nats. The convolution has no SSM recurrent state yet matches a full state space model.

3.2 Recurrent Inference

We evaluate all three models using their recurrent (autoregressive) form on 4096-token windows from The Stack validation set, averaging over 8 windows. Table 2 shows per-segment losses.

Table 2: Recurrent inference on The Stack (4096 tokens, 8 windows). Loss in nats by position segment.

Model	0–1K	1–2K	2–3K	3–4K	Overall
Mamba (no Heb)	1.98	1.87	2.05	1.99	1.97
Conv+Heb	1.46	1.35	1.47	1.44	1.43
Mamba+Heb	1.30	1.40	1.51	1.25	**1.37**

Both memory-augmented models outperform Mamba alone by 0.5+ nats, confirming that the Hebbian memory drives the improvement. Mamba+Heb edges out Conv+Heb by 0.06 nats in recurrent mode—Mamba's recurrence provides a small benefit for autoregressive generation over long contexts, though the gap is minor compared to the 0.60-nat contribution of the memory itself. All models show stable loss across sequence length.

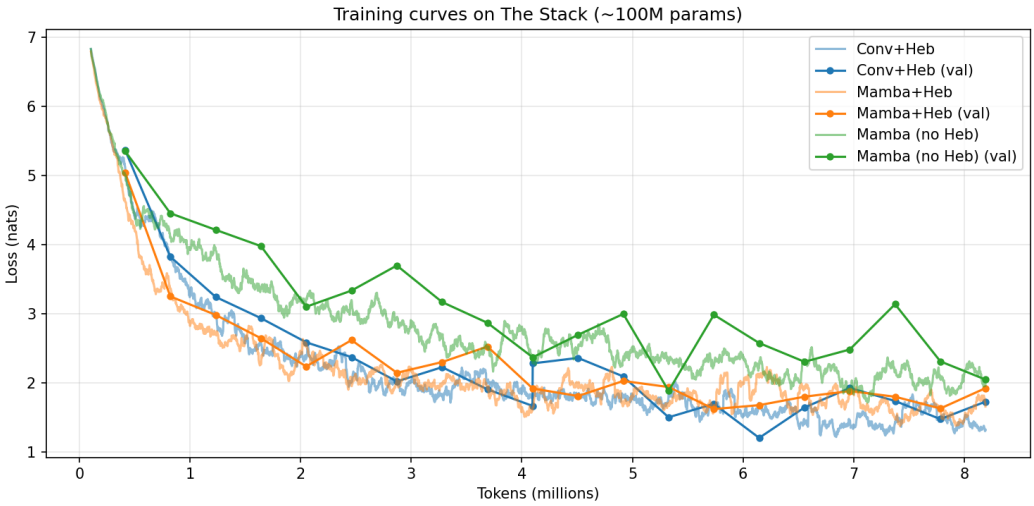

Figure 2: Training curves on The Stack ($\sim$100M params, $\sim$8M tokens). Solid lines show smoothed training loss; markers show validation loss. Conv+Heb tracks Mamba+Heb throughout training, while Mamba without Hebbian memory lags by $\sim$0.7 nats.

3.3 Learned Memory Parameters

The Conv+Heb model learns per-layer memory strength α and decay γ. Figure 3 shows the learned values after 4000 steps.

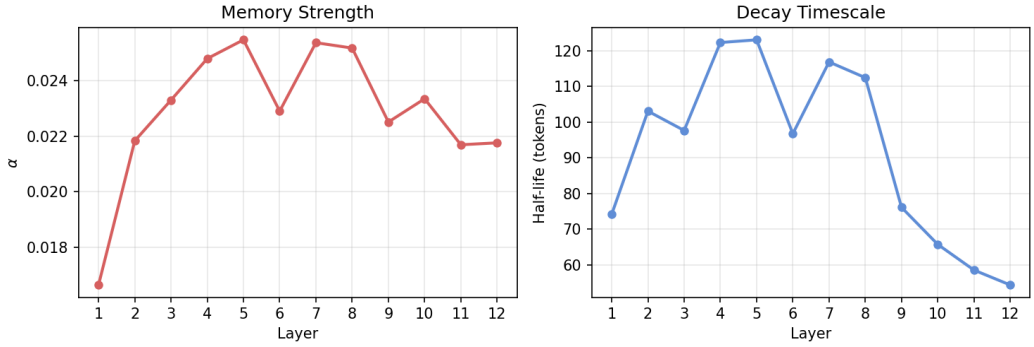

Figure 3: Learned per-layer parameters in the 100M Conv+Heb model. **Left**: Memory strength α peaks in middle layers ($\sim$0.025) and drops at boundaries ($\sim$0.017). **Right**: Decay half-life ($\log 2/\log(1/\gamma)$) shows a timescale hierarchy—early layers retain memory longer ($\sim$120 tokens), later layers decay faster ($\sim$55 tokens).

The α profile suggests middle layers perform the bulk of associative reasoning, while boundary layers focus on local features (early) and decoding (late). The decay hierarchy means early layers build broad context while later layers focus on recent associations.

3.4 FineWeb Scaling

To test generalization beyond code, we train a 124M-parameter Conv+Heb model on FineWeb [Penedo et al., 2024], a large-scale English web corpus, and compare against the nanoGPT 124M transformer baseline (3.28 nats).

Baseline The target is 3.28 nats, the validation loss achieved by Karpathy's nanoGPT 124M reproduction in llm.c [Karpathy, 2024]—a GPT-2-architecture transformer (12 layers, D=768, 12 heads)

trained with AdamW on ∼10B tokens of FineWeb with GPT-2 tokenization (vocab 50,304) and sequence length 1024.

Our setup Conv+Heb uses 18 layers, D=768, expand factor 2, d_{conv}=4, with learned per-layer α (init 0.03) and γ (init 0.99). Training uses AdamW (β_1=0.9, β_2=0.95), cosine schedule with peak lr=6×10^{-4} decaying to 6×10^{-5}, 500-step warmup, weight decay 0.1, gradient clipping at 1.0, and bf16 autocast. We train for 19,000 steps with 524K tokens per step (∼10B tokens total), matching the baseline's token budget. Training was done on a single H100 GPU with `torch.compile`.

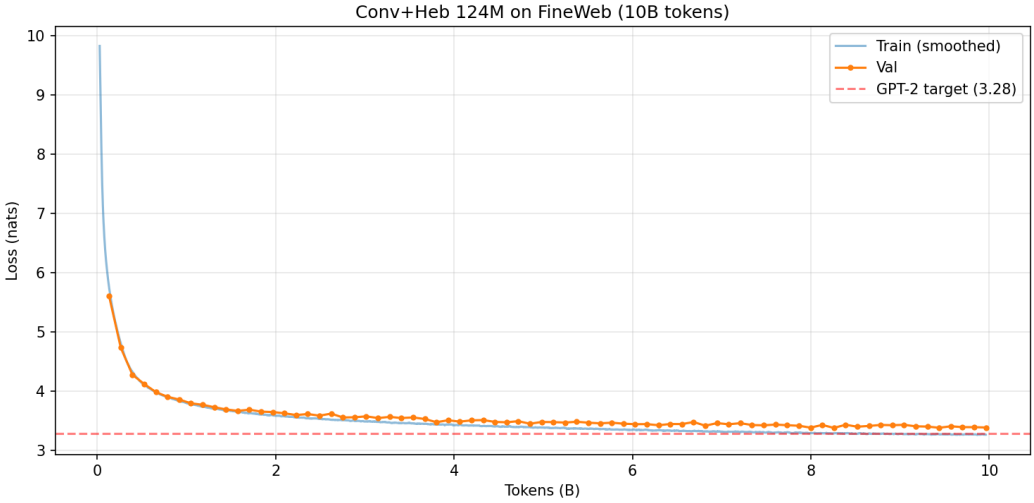

Figure 4: Conv+Heb (124M params) on FineWeb. The dashed line shows the nanoGPT 124M baseline (3.28 nats). Final validation loss: 3.39 nats, a gap of 0.11 nats.

Conv+Heb reaches 3.39 nats versus the baseline of 3.28 nats—a gap of 0.11 nats. Both models use AdamW with comparable training recipes and the same token budget. The 0.11-nat difference is consistent with the small-scale behavior of linear attention variants (RetNet, GLA, DeltaNet), which show similar gaps at 124M–350M parameters but close and even surpass transformers at larger scales—RetNet at 6.7B achieves lower perplexity than its transformer baseline [Sun et al., 2023]. The gap likely reflects scale, not a fundamental limitation of the memory mechanism.

4 Discussion

Hebbian memory as the dominant mechanism The +0.71 nat gap between Mamba and Mamba+Heb versus the 0.03-nat gap between Mamba+Heb and Conv+Heb points to a clear decomposition: the memory is doing the work. The outer-product matrix is content-addressable—retrieval is by similarity to the query, not by temporal position—making it fundamentally different from SSM recurrence, which compresses history into a fixed-size state vector.

This echoes complementary learning systems in neuroscience [McClelland et al., 1995]: neocortex for slow feature learning, hippocampus for fast associative binding. The Hebbian memory plays the hippocampal role.

Connection to fast weight programmers and linear attention Our memory is an instance of fast weight programming [Schmidhuber, 1992, Ba et al., 2016, Schlag et al., 2021]. Linear attention [Katharopoulos et al., 2020] maintains the same outer-product matrix without decay; adding learnable decay connects to RetNet [Sun et al., 2023] and GLA [Yang et al., 2024]. Our contribution is not the mechanism itself, but the finding that *it alone, paired with a trivial backbone, suffices for competitive language modeling*.

Limitations Our 100M-parameter models are trained on only ∼8M tokens (The Stack), which is significantly undertrained by modern standards. The FineWeb experiment uses a full 10B-token

budget but is limited to 124M parameters. Scaling to larger models and longer training would strengthen the comparison.

5 Related Work

State space models S4 [Gu et al., 2022] introduced structured state spaces for sequence modeling. Mamba [Gu and Dao, 2024] added input-dependent selection, achieving strong performance on language modeling. Mamba-2/SSD [Dao and Gu, 2024] revealed the duality between SSMs and structured attention, enabling efficient multi-head implementations. Our work shows that for models augmented with associative memory, the SSM backbone can be replaced with a simpler alternative.

Associative memory and fast weights Our Hebbian memory follows the fast weight lineage [Schmidhuber, 1992, Ba et al., 2016, Schlag et al., 2021] and is closely related to linear attention [Katharopoulos et al., 2020], RetNet [Sun et al., 2023], GLA [Yang et al., 2024], and RWKV [Peng et al., 2023]. Hopfield networks [Hopfield, 1982, Ramsauer et al., 2021] provide an energy-based alternative. Our contribution is showing that these mechanisms make the surrounding architecture largely redundant.

6 Conclusion

A 4-token depthwise convolution paired with Hebbian outer-product memory matches a 100M-parameter Mamba+Heb on code language modeling—with no recurrence. Removing the memory costs 0.71 nats. Replacing the SSM with a trivial convolution costs 0.03 nats in parallel training and 0.06 nats in autoregressive inference. On FineWeb at 124M parameters, Conv+Heb reaches within 0.11 nats of nanoGPT 124M—a gap consistent with other linear attention variants that closes at larger scales.

The backbone is not irrelevant—the 0.11-nat gap on FineWeb shows it contributes something, likely precise short-range token interactions. But it is not the dominant mechanism. The bulk of language modeling performance comes from associative storage, and the complexity of modern SSMs and attention is disproportionate to their contribution. Future work should focus on the memory, not the backbone.

References

Jimmy Ba, Geoffrey E Hinton, Volodymyr Mnih, Joel Z Leibo, and Catalin Ionescu. Using fast weights to attend to the recent past. *Advances in Neural Information Processing Systems*, 2016.

Tri Dao and Albert Gu. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. In *International Conference on Machine Learning*, 2024.

Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. In *International Conference on Machine Learning*, 2024.

Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. *International Conference on Learning Representations*, 2022.

Donald O Hebb. The organization of behavior. *Wiley*, 1949.

John J Hopfield. Neural networks and physical systems with emergent collective computational abilities. *Proceedings of the National Academy of Sciences*, 79(8):2554–2558, 1982.

Andrej Karpathy. llm.c: LLM training in simple, raw C/CUDA, 2024. `https://github.com/karpathy/llm.c`.

Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *International Conference on Machine Learning*, 2020.

James L McClelland, Bruce L McNaughton, and Randall C O'Reilly. Why there are complementary learning systems in the hippocampus and neocortex: insights from the successes and failures of connectionist models of learning and memory. *Psychological Review*, 102(3):419, 1995.

Guilherme Penedo, Hynek Kydlíček, Alessandro Cappelli, Hector Saez de Ocariz Borde, et al. The FineWeb datasets: Decanting the web for the finest text data at scale. *arXiv preprint arXiv:2406.17557*, 2024.

Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Huanqi Cao, Xin Cheng, Michael Chung, Matteo Grella, et al. RWKV: Reinventing RNNs for the transformer era. In *Findings of the Association for Computational Linguistics: EMNLP*, 2023.

Hubert Ramsauer, Bernhard Schäfl, Johannes Lehner, Philipp Seidl, Michael Widrich, Lukas Gruber, Markus Holzleitner, Miroslav Pavlović, Geir Kjetil Sandve, Thomas Unterthiner, et al. Hopfield networks is all you need. In *International Conference on Learning Representations*, 2021.

Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In *International Conference on Machine Learning*, 2021.

Jürgen Schmidhuber. Learning to control fast-weight memories: An alternative to dynamic recurrent networks. *Neural Computation*, 4(1):131–139, 1992.

Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models. *arXiv preprint arXiv:2307.08621*, 2023.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

Songlin Yang, Bailin Wang, Yikang Shen, Rameswar Panda, and Yoon Kim. Gated linear attention transformers with hardware-efficient training. *International Conference on Machine Learning*, 2024.